

Отчет по лабораторной работе №6

Администрирование ОС Linux. SELinux

Барето Вилиан Мануел

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Выводы	13
	Список литературы	14

Список иллюстраций

2.1	проверка режима работы SELinux	6
2.2	Проверка работы Apache	6
2.3	Контекст безопасности Apache	7
2.4	Состояние переключателей SELinux	7
2.5	Статистика по политике	8
2.6	Типы поддиректорий	8
2.7	Типы файлов	8
2.8	Создание файла	9
2.9	Контекст файла	9
2.10	Отображение файла	9
2.11	Изучение справки по команде	10
2.12	Изменение контекста	10
2.13	Отображение файла	10
2.14	Попытка прочесть лог-файл	11
2.15	Изменение файла	11
2.16	Попытка прослушивания другого порта	11
2.17	Проверка лог-файлов	11
2.18	Проверка лог-файлов	11
2.19	Проверка портов	12
2.20	Перезапуск сервера	12
2.21	Проверка порта 81	12
2.22	Удаление файла	12

Список таблиц

1 Цель работы

Развить навыки администрирования ОС linux. Получить практическое знакомство с SELinux1. Проверить работу SELinux на практике совместно с веб-сервером Apache.

2 Выполнение лабораторной работы

Вошла в систему под своей учетной записью. Убедилась, что SELinux работает в режиме enforcing политики targeted с помощью getenforce и sestatus

```
wbarreto@Willian:~$ sestatus
SELinux status:                enabled
SELinuxfs mount:              /sys/fs/selinux
SELinux root directory:      /etc/selinux
Loaded policy name:           targeted
Current mode:                 enforcing
Mode from config file:       enforcing
Policy MLS status:           enabled
Policy deny_unknown status:   allowed
Memory protection checking:   actual (secure)
Max kernel policy version:    33
wbarreto@Willian:~$
```

Рис. 2.1: проверка режима работы SELinux

Запускаю сервер apache, далее обращаюсь с помощью браузера к веб-серверу, запущенному на компьютере, он работает, что видно из вывода команды service httpd status

```
wbarreto@Willian:~$ sudo systemctl start httpd
wbarreto@Willian:~$ sudo systemctl enable httpd
Created symlink '/etc/systemd/system/multi-user.target.wants/httpd.service' → '/usr/lib/systemd/system/httpd.service'.
wbarreto@Willian:~$ service httpd status
```

Рис. 2.2: Проверка работы Apache

С помощью команды `ps auxZ | grep httpd` нашла веб-сервер Apache в списке процессов. Его контекст безопасности - `httpd_t`

```
wbarreto@Willian:~$ ps auxZ | grep httpd
system_u:system_r:httpd_t:s0 root 11733 0.0 0.1 19140 11264 ? Ss 21:28 0:00 /usr/sbin/httpd -D
FOREGROUND
system_u:system_r:httpd_t:s0 apache 11734 0.0 0.0 18796 5320 ? S 21:28 0:00 /usr/sbin/httpd -D
FOREGROUND
system_u:system_r:httpd_t:s0 apache 11735 0.0 0.1 1109288 8020 ? Sl 21:28 0:00 /usr/sbin/httpd -D
FOREGROUND
system_u:system_r:httpd_t:s0 apache 11738 0.0 0.1 978088 7792 ? Sl 21:28 0:00 /usr/sbin/httpd -D
FOREGROUND
system_u:system_r:httpd_t:s0 apache 11787 0.0 0.1 978088 7792 ? Sl 21:28 0:00 /usr/sbin/httpd -D
FOREGROUND
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 wbarreto 12278 0.0 0.0 227704 2248 pts/1 S+ 21:35 0:00 grep
--color=auto httpd
wbarreto@Willian:~$
```

Рис. 2.3: Контекст безопасности Apache

Просмотрела текущее состояние переключателей SELinux для Apache

```
wbarreto@Willian:~$ sestatus -b httpd
SELinux status: enabled
SELinuxfs mount: /sys/fs/selinux
SELinux root directory: /etc/selinux
Loaded policy name: targeted
Current mode: enforcing
Mode from config file: enforcing
Policy MLS status: enabled
Policy deny_unknown status: allowed
Memory protection checking: actual (secure)
```

Рис. 2.4: Состояние переключателей SELinux

Просмотрела статистику по политике с помощью команды seinfo. Множество пользователей - 8, ролей - 39, типов - 5135.

Classes:	135	Permissions:	457
Sensitivities:	1	Categories:	1024
Types:	5169	Attributes:	259
Users:	8	Roles:	15
Booleans:	358	Cond. Expr.:	390
Allow:	65633	Neverallow:	0
Auditallow:	176	Dontaudit:	8703
Type_trans:	271851	Type_change:	94
Type_member:	37	Range_trans:	5931
Role allow:	40	Role_trans:	417
Constraints:	70	Validatetrans:	0
MLS Constrains:	72	MLS Val. Tran:	0
Permissives:	1	Polcap:	6
Defaults:	7	Typebounds:	0
Allowxperm:	0	Neverallowxperm:	0
Auditallowxperm:	0	Dontauditxperm:	0
Ibendportcon:	0	Ibpkeycon:	0
Initial SIDs:	27	Fs_use:	35
Genfscon:	109	Portcon:	665
Netifcon:	0	Nodecon:	0

wakutaipa@mwakutaipa ~]\$

Рис. 2.5: Статистика по политике

Типы поддиректорий, находящихся в директории /var/www, с помощью команды `ls -lZ /var/www` следующие: владелец - root, права на изменения только у владельца. Файлов в директории нет

```
wbarretogWillian:~$ ls -lZ /var/www
total 0
drwxr-xr-x. 2 root root system_u:object_r:httpd_sys_script_exec_t:s0  6 dez 10 03:00 cgi-bin
drwxr-xr-x. 2 root root system_u:object_r:httpd_sys_content_t:s0    23 abr 29 21:45 html
wbarretogWillian:~$
```

Рис. 2.6: Типы поддиректорий

В директории /var/www/html нет файлов.

```
wbarretogWillian:~$ ls -lZ /var/www/html
total 0
wbarretogWillian:~$
```

Рис. 2.7: Типы файлов

Создать файл может только суперпользователь, поэтому от его имени создаем файл `touch.html` со следующим содержанием:


```
<html>
<body>test</body>
</html>
```

```
wbarreto@willian:~$ sudo touch /var/www/html/test.html
[sudo] senha para wbarreto:
wbarreto@willian:~$ sudo gedit /var/www/html/test.html
```

Рис. 2.8: Создание файла

Проверяю контекст созданного файла. По умолчанию это httpd_sys_content_t.

```
wbarreto@willian:~$ ls -lZ /var/www/html
total 0
wbarreto@willian:~$ sudo touch /var/www/html/test.html
[sudo] senha para wbarreto:
wbarreto@willian:~$ sudo gedit /var/www/html/test.html
```

Рис. 2.9: Контекст файла

Обращаюсь к файлу через веб-сервер, введя в браузере адрес <http://127.0.0.1/test.html>. Файл был успешно отображён.

```
wbarreto@willian:~$ echo "<html> <body>Test test)</body> </html>" | sudo tee /var/www/html/test.html
<html> <body>Test test)</body> </html>
wbarreto@willian:~$
```

Рис. 2.10: Отображение файла

Изучила справку `man httpd_selinux`. Рассмотрим полученный контекст детально. Так как по умолчанию пользователи CentOS являются свободными от типа (`unconfined` в переводе с англ. означает свободный), созданному нами файлу `test.html` был сопоставлен SELinux, пользователь `unconfined_u`. Это первая часть контекста. Далее политика ролевого разделения доступа RBAC используется процессами, но не файлами, поэтому роли не имеют никакого значения для файлов. Роль `object_r` используется по умолчанию для файлов на «постоянных» носителях и на сетевых файловых системах. (В директории `/proc` файлы, относящиеся к процессам, могут иметь роль `system_r`. Если активна политика MLS, то могут использоваться и другие роли. Данный случай мы рассматривать не будем, как

и предназначение :s0). Тип `httpd_sys_content_t` позволяет процессу `httpd` получить доступ к файлу. Благодаря наличию последнего типа мы получили доступ к файлу при обращении к нему через браузер.

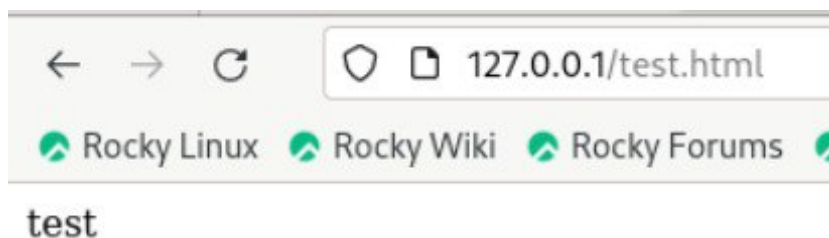


Рис. 2.11: Изучение справки по команде

Изменяю контекст файла `test.html` с `httpd_sys_content_t` на любой другой, к которому процесс `httpd` не должен иметь доступа, например, на `samba_share_t`: `chcon -t samba_share_t /var/www/html/test.html ls -Z /var/www/html/test.html`. Контекст действительно поменялся.

Изменение контекста

Рис. 2.12: Изменение контекста

При попытке отображения файла в браузере получаем сообщение об ошибке.

Отображение файла

Рис. 2.13: Отображение файла

файл не был отображён, хотя права доступа позволяют читать этот файл любому пользователю, потому что установлен контекст, к которому процесс `httpd` не должен иметь доступа. Просматриваю log-файлы веб-сервера Apache и системный лог-файл: `tail /var/log/messages`. Если в системе окажутся запущенными

процессы `setroubleshootd` и `audtd`, то вы также сможете увидеть ошибки, аналогичные указанным выше, в файле `/var/log/audit/audit.log`.

Попытка прочесть лог-файл

Рис. 2.14: Попытка прочесть лог-файл

Чтобы запустить веб-сервер Apache на прослушивание TCP-порта 81 (а не 80, как рекомендует IANA и прописано в `/etc/services`) открываю файл `/etc/httpd/httpd.conf` для изменения. Нахожу строчку `Listen 80` и заменяю её на `Listen 81`.

Изменение файла

Рис. 2.15: Изменение файла

Выполняю перезапуск веб-сервера Apache. Произошёл сбой, потому что порт 80 для локальной сети, а 81 нет.

Попытка прослушивания другого порта

Рис. 2.16: Попытка прослушивания другого порта

Проанализировала лог-файлы: `tail -nl /var/log/messages`

Проверка лог-файлов

Рис. 2.17: Проверка лог-файлов

Запись появилась в файлу `error_log`.

Проверка лог-файлов

Рис. 2.18: Проверка лог-файлов

Выполняю команду `semanage port -a -t http_port_t -p tcp 81`. После этого проверяю список портов командой `semanage port -l | grep http_port_t`. Порт 81 появился в списке

Проверка портов

Рис. 2.19: Проверка портов

Перезапускаю сервер Apache

Перезапуск сервера

Рис. 2.20: Перезапуск сервера

Теперь он работает, ведь мы внесли порт 81 в список портов `httpd_port_t`. Возвращаю в файле `/etc/httpd/httpd.conf` порт 80, вместо 81. Проверяю, что порт 81 удален, это правда.

Проверка порта 81

Рис. 2.21: Проверка порта 81

Далее удаляю файл `test.html`, проверяю, что он удален

Удаление файла

Рис. 2.22: Удаление файла

3 Выводы

При выполнении данной лабораторной работы были развиты навыки администрирования ОС Linux, получено первое практическое знакомство с технологией SELinux и проверена работа SELinux на практике совместно с веб-сервером Apache.

Список литературы

::: {#refs}